A gm/Id Methodology Design Tool for Analog CMOS Circuit Synthesis: Application to the Folded-Cascode Class-AB Amplifier

Phil Alcorn
Department of Electrical Engineering
Tarleton State University
Stephenville, TX
palcorn2000@gmail.com

***Abstract*—This paper presents a Python-based simulation pipeline that automates the gm/Id transistor sizing methodology for CMOS analog circuits. The tool characterizes NMOS and PMOS transistors across channel length, gate-to-source voltage, drain-to-source voltage, and bulk-to-source voltage using LTspice operating-point sweeps, stores the results in a four-dimensional look-up table (LUT), and provides a query interface for transistor sizing. The design flow is demonstrated by applying the extracted LUTs to the sizing of a folded-cascode class-AB operational amplifier, walking through each transistor group from specification to final width.**

***Index Terms*—gm/Id methodology, CMOS amplifier design, LTspice automation, look-up table, folded-cascode amplifier, class-AB output stage, transistor sizing**

I. Introduction

Introductory analog circuit courses use the square-law MOSFET model to derive sizing equations. The square-law predicts that in saturation

$$I_D = \frac{1}{2}\mu_n C_{ox}\frac{W}{L}(V_{GS} - V_{th})^2, \tag{1}$$

and sizing a transistor means solving (1) for W/L given a target I_D and an assumed overdrive $V_{ov} = V_{GS} - V_{th}$. In practice, modern SPICE models include effects that make the real device behave differently from this simple equation, so a transistor sized by hand from (1) will not match its simulated operating point. Designers then iterate between hand calculations and simulation until the numbers converge, which is time-consuming.

The gm/Id methodology [1] bypasses this iteration by replacing the square-law with data taken directly from the SPICE model. Rather than computing W from an equation, the designer looks up how much current a given width carries at the desired bias condition and scales accordingly. Because the data already include all of the model's non-ideal behavior, no iteration is needed. Murmann's lecture notes [2] and Tiwari's design slides [3] provide accessible introductions to the methodology; Allen and Holberg [4] give a thorough treatment of the circuit topologies used in the design example.

This work implements the gm/Id workflow in Python on top of PyLTSpice [5], enabling automated parametric sweeps, structured data extraction, and an interactive query interface. Section II explains the gm/Id concept. Section III describes the software. Section IV covers the characterization sweep. Section V explains the LUT and how sizing queries work. Section VI applies the tool to a complete folded-cascode class-AB amplifier design.

II. The gm/Id Design Methodology

A. Transconductance Efficiency

The transconductance g_m of a MOSFET is defined as the change in drain current per change in gate voltage, $g_m = \partial I_D/\partial V_{GS}$. Dividing by the quiescent drain current gives the *transconductance efficiency*:

$$\frac{g_m}{I_D}\ \left[\mathrm{V}^{-1}\right]. \tag{2}$$

This ratio answers the question: *how much small-signal gain does this transistor produce per unit of DC current?* A high g_m/I_D means the device is power-efficient; a low g_m/I_D means the device is fast but draws more current for the same g_m.

From the square-law, $g_m = 2I_D/V_{ov}$, so

$$\left.\frac{g_m}{I_D}\right|_{\text{sq.law}} = \frac{2}{V_{ov}}. \tag{3}$$

As V_{GS} increases (device driven harder into strong inversion), V_{ov} grows and g_m/I_D falls. As V_{GS} approaches V_{th} the device enters weak inversion (subthreshold), where I_D is exponential and g_m/I_D reaches its maximum of roughly 25–$35\,\mathrm{V}^{-1}$ for a typical process.

In reality, g_m/I_D is a smooth, monotonically decreasing function of V_{GS} that does not follow (3) exactly. The LUT captures this real curve directly from simulation data.

B. What the gm/Id Value Tells You

Choosing a g_m/I_D target is equivalent to choosing where on the weak-inversion to strong-inversion continuum the transistor will be biased:

- $g_m/I_D \approx 25\,\mathrm{V}^{-1}$: weak inversion. Maximum power efficiency. The transistor needs a small V_{GS} above V_{th} and draws very little current, but it is slow and V_{dsat} is large (poor output swing).
- $g_m/I_D \approx 15\,\mathrm{V}^{-1}$: moderate inversion. Good balance of efficiency and speed. A practical starting point for most signal-path transistors.
- $g_m/I_D \approx$ 5–10 V^{-1}: strong inversion. Low efficiency, but the device is fast and V_{dsat} is small, which is useful for output-stage transistors where output swing matters.

Once a g_m/I_D target is chosen, the LUT gives the drain current per unit width (I_D/W) at that operating point. Knowing the required I_D from the circuit specification, the width follows directly:

$$W = \frac{I_{D,\text{required}}}{(I_D/W)_{\text{LUT}}}. \tag{4}$$

No iteration is needed.

C. Key Design Variables in the LUT

For each bias point the LUT stores the following quantities (per unit width, for $W_{\text{ref}} = 10\,\mu$m):

- I_D, g_m, g_{ds} — drain current, transconductance, and output conductance from the SPICE operating-point solver.
- g_m/I_D — transconductance efficiency (computed).
- I_D/W — current density (computed from I_D and W_{ref}).
- g_m/g_{ds} — intrinsic gain; useful for estimating A_v of a single-transistor stage.
- $f_T = g_m/(2\pi C_{gg})$ — transition frequency; a measure of how fast the transistor is.
- V_{th}, V_{dsat} — threshold and saturation voltage.

III. Software Architecture

The pipeline is implemented in Python and uses PyLTSpice [5] to drive LTspice. Four modules handle the workflow.

`spice.py` wraps the PyLTSpice simulation runner. It converts the `.asc` schematic to a netlist, updates `.param` values for each sweep point, and runs up to 20 LTspice processes in parallel.

`callbacks.py` runs after each simulation finishes. It reads the `.log` operating-point data and the `.raw` waveform file, and bundles everything into a `SimResult` object keyed by device name.

`parser.py` converts a list of `SimResult` objects into structured `ParsedSimResult` objects containing the traces, measurements, and device parameters the designer requested.

`characterization.py` coordinates the full characterization sweep, assembles results into four-dimensional NumPy arrays indexed by $(L, V_{GS}, V_{DS}, V_{SB})$, and saves them to `.npz` files. **`query_lut.py`** loads a saved LUT and answers sizing queries via command-line arguments or an interactive terminal prompt.

IV. Device Characterization Sweep

A. Testbench Topology

The characterization schematic (`nmos_char.asc`) contains one NMOS (M1) and one PMOS (M2) transistor. Both are characterized in a single operating-point simulation by mirroring the NMOS bias about the supply rail for the PMOS:

$$V_{g,p} = V_{DD} - V_{gs}, \tag{5}$$
$$V_{d,p} = V_{DD} - V_{ds}, \tag{6}$$
$$V_{b,p} = V_{DD} - V_b. \tag{7}$$

This means that sweeping (V_{gs}, V_{ds}) for the NMOS simultaneously sweeps the intrinsic (V_{SG}, V_{SD}) of the PMOS, so one Python loop produces LUTs for both device types.

B. Sweep Parameters

- $L \in \{0.35, 0.70, 1.00, 2.00\}\,\mu$m
- V_{GS} from 0 to 5 V in 50 mV steps (101 points)
- V_{DS} from 0 to 5 V in 50 mV steps (101 points)
- $V_{SB} \in \{0, 0.45, 0.90\}$ V
- Reference width $W_{\text{ref}} = 10\,\mu$m

This produces $4 \times 101 \times 101 \times 3 = 122\,412$ operating-point simulations run in parallel across 20 LTspice worker processes.

C. Extracted Fields

TABLE I
Operating-Point Parameters Extracted Per Device

Symbol	SPICE name	Description
I_D	`Id`	Drain current
g_m	`Gm`	Transconductance
g_{ds}	`Gds`	Output conductance ($1/r_o$)
g_{mb}	`Gmb`	Body transconductance
C_{gg}	`dQgdVgb` + overlap	Total gate capacitance
V_{th}	`Vth`	Threshold voltage
V_{dsat}	`Vdsat`	Minimum V_{DS} for saturation

V. Look-Up Table and Sizing Queries

A. LUT Structure

After all simulations complete, the results are assembled into four-dimensional arrays of shape $(n_L, n_{Vgs}, n_{Vds}, n_{Vsb})$. The following derived quantities are appended:

$$\frac{g_m}{I_D}, \quad \frac{I_D}{W}, \quad \frac{g_m}{g_{ds}}, \quad f_T = \frac{g_m}{2\pi C_{gg}}. \tag{8}$$

Both NMOS and PMOS LUTs are saved to compressed NumPy archives (`nmos_lut.npz` and `pmos_lut.npz`).

B. Query Modes

`query_lut.py` supports two usage modes.

Bias mode: Provide (L, V_{GS}, V_{DS}) and get all operating-point quantities back. Use this to check what a transistor looks like at a specific bias point.

```
python query_lut.py --l 1.0 --vgs 0.8 --vds
    0.9
```

Design mode: Provide a g_m/I_D target, L, and V_{DS}, and optionally a target g_m or I_D. The tool finds the V_{GS} that hits the efficiency target, reads the current density from the LUT, and returns the required width using (4).

```
python query_lut.py --l 1.0 --gmid 15 --vds
    0.9 \
  --gm 628e-6
```

The design mode output includes V_{GS}, I_D, W, g_m/g_{ds}, V_{dsat}, and f_T — all the values needed to verify a design decision and move on to the next transistor.

VI. Application: Folded-Cascode Class-AB Amplifier

A. Topology Overview

The folded-cascode amplifier is a single-stage op-amp topology that achieves high DC gain without the voltage headroom problem of an all-NMOS or all-PMOS cascode [4], [6]. Its core idea is to take the output current of the differential input pair and "fold" it into cascode transistors of the opposite polarity, as shown in Fig. 1.

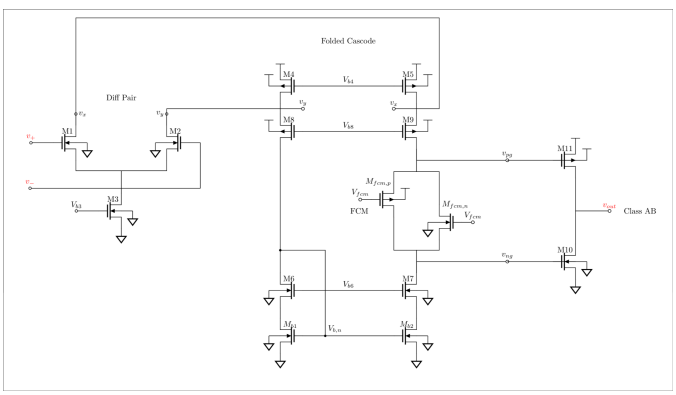

Fig. 1. Folded-cascode class-AB operational amplifier showing the differential input pair (M1–M3), PMOS folding current sources (M4, M5), cascode stacks (M6–M9), floating current mirror (FCM), and class-AB push-pull output stage (M10, M11).

The transistors and their roles are:

- **M1, M2** (NMOS differential pair): The input transistors. Their g_m determines the GBW of the amplifier.
- **M3** (NMOS tail current source): Sets the total quiescent current through M1 and M2.
- **M4, M5** (PMOS folding current sources): Supply a fixed current into each folding node. The difference between this current and M1/M2's drain current becomes the signal current in the output branch.
- **M6, M7** (NMOS cascode transistors): Stacked below the output node to increase the NMOS output resistance.
- **M8, M9** (PMOS cascode transistors): Stacked above the output node to increase the PMOS output resistance.
- **M10, M11** (NMOS/PMOS class-AB output pair): A push-pull output stage driven by the folded-cascode output. M10 sources current when the output swings low; M11 sinks current when the output swings high.

B. Gain and Output Resistance

The open-loop voltage gain of the folded-cascode first stage is

$$A_v = g_{m1} \cdot R_{out}, \tag{9}$$

where R_{out} is the total resistance at the output node. Looking down into the NMOS cascode stack the resistance is approximately

$$R_{out,n} \approx \frac{g_{m6}}{g_{ds6}} \cdot \frac{1}{g_{ds1}}, \tag{10}$$

and looking up into the PMOS cascode stack

$$R_{out,p} \approx \frac{g_{m8}}{g_{ds8}} \cdot \frac{1}{g_{ds4}}. \tag{11}$$

The total output resistance is $R_{out} = R_{out,n} \| R_{out,p}$, and the gain follows from (9). Note that g_m/g_{ds} is the *intrinsic gain* of a single transistor and is available directly from the LUT under the field `gm_gds`. A target of $A_v > 60\,\text{dB}$ (i.e., $> 1000\,\text{V/V}$) requires that both cascode stacks contribute sufficient intrinsic gain.

C. Gain-Bandwidth Product

Assuming the load capacitance C_L creates the dominant pole at the output node, the unity-gain bandwidth is

$$\text{GBW} = \frac{g_{m1}}{2\pi C_L}. \tag{12}$$

This is the most important equation for starting the design because it directly gives the required g_m of the input pair:

$$g_{m1,\text{req}} = 2\pi \cdot \text{GBW} \cdot C_L. \tag{13}$$

D. Slew Rate

The slew rate is limited by how quickly the circuit can charge or discharge C_L. At quiescent, the output branch carries current I_{out}. Under a large input step, all of the tail current is steered to one side, giving

$$\text{SR} = \frac{I_{tail}}{C_L}. \tag{14}$$

For the class-AB output stage, the peak output current can be much larger than the quiescent current, which is the advantage of the class-AB topology over a simple class-A buffer.

E. Current Budget

Before querying the LUT for any transistor, it helps to establish the current budget — how much current flows through each branch at quiescent. Using the convention that I_{D1} is the drain current of each input transistor:

- Tail current source M3: $I_{tail} = 2I_{D1}$
- Each side of the differential pair (M1 or M2): I_{D1}
- Each PMOS folder (M4 or M5): $I_{D4} = I_{D1} + I_{cas}$, where I_{cas} is the quiescent current in the output cascode stack. A common choice is $I_{cas} = I_{D1}$, giving $I_{D4} = 2I_{D1}$.
- Each output cascode transistor (M6–M9): $I_{cas} = I_{D1}$
- Output stage (M10, M11): quiescent current I_q set separately by the class-AB bias circuit.

F. Step-by-Step Sizing Procedure

The procedure below assumes the following example specifications. Substitute your own values in the bracketed fields.

TABLE II
EXAMPLE DESIGN SPECIFICATIONS

Parameter	Symbol	Value
Supply voltage	V_{DD}	3.3 V
Load capacitance	C_L	10 pF
GBW target	GBW	10 MHz
DC gain target	A_v	> 60 dB
Slew rate taret	SR	> 5 V/μs
Output swing	—	± 1 V from mid-rail

1) Step 1: Compute Required Input-Pair Transconductance: Apply (13) to find the transconductance the input pair must provide:

$$g_{m1,\text{req}} = 2\pi \times 10\,\text{MHz} \times 10\,\text{pF} \approx 628\,\mu\text{A/V}. \tag{15}$$

This number is fixed by the specification and does not change regardless of which transistor is chosen. Everything else follows from it.

2) Step 2: Choose gm/Id and L for the Input Pair (M1, M2): Look at the I_D/W **vs. gm/Id** plot (Fig. 2) and the f_T **vs. gm/Id** plot (Fig. 5) to pick an operating point for M1/M2.

- Choose $g_m/I_D = 15\,\text{V}^{-1}$ (moderate inversion — good balance of efficiency and bandwidth).
- Choose $L = 1.0\,\mu$m (longer channel for better intrinsic gain; check that the `gm_gds` value is large enough after the query).
- Assume $V_{DS1} \approx 0.9$ V for the query. This is a reasonable mid-supply estimate; you will verify it fits once the full bias point is known.

Run the design-mode query, passing the required g_m:

```
python query_lut.py \
  --l 1.0 --gmid 15 --vds 0.9 \
  --gm 628e-6
```

What to record from the output:

- V_{GS1} — you will need this to set the bias network.
- I_{D1} — the quiescent drain current of each input transistor.
- $W_{1,2}$ — the transistor width the tool computed from (4).
- V_{dsat1} — needed for the output swing headroom check in Step 8.

3) Step 3: Size the Tail Current Source (M3): The tail source must supply $I_{tail} = 2I_{D1}$.

- Use NMOS. Choose a long channel ($L = 2.0\,\mu$m) to maximize r_o, which increases the common-mode rejection ratio.
- The tail source is not in the signal path, so its g_m/I_D is chosen to keep V_{dsat} small (preserving headroom at the bottom of the supply) while ensuring saturation. A value of $g_m/I_D \approx 10\,\text{V}^{-1}$ is a reasonable starting point.
- Assume $V_{DS,tail} \approx 0.3$ V — just enough to keep it in saturation.

```
python query_lut.py \
  --l 2.0 --gmid 10 --vds 0.3 \
  --id <2 * ID1 from Step 2>
```

Record V_{GS3}, W_3, and V_{dsat3}.

4) Step 4: Size the PMOS Folding Current Sources (M4, M5): Each PMOS folder supplies $I_{D4} = 2I_{D1}$ (one I_{D1} for the differential pair branch and one I_{D1} for the output cascode branch).

- Use the PMOS LUT (`--path char_data/pmos_lut.npz`).
- Choose $L = 1.0\,\mu$m.
- Choose $g_m/I_D \approx 12\,\text{V}^{-1}$ (moderate inversion). The PMOS folders are current sources, not signal transistors, but their r_o appears in $R_{out,p}$ so a longer channel or moderate inversion is preferred.
- Assume $V_{SD4} \approx 0.9$ V (note: for PMOS the tool uses V_{SG} and V_{SD} internally, but the command-line arguments `--vgs` and `--vds` refer to the intrinsic quantities, so enter positive magnitudes the same way as for NMOS).

```
python query_lut.py \
  --path char_data/pmos_lut.npz \
  --l 1.0 --gmid 12 --vds 0.9 \
  --id <2 * ID1>
```

Record V_{SG4}, $W_{4,5}$, and V_{dsat4}.

5) Step 5: Size the NMOS Cascode Transistors (M6, M7): M6/M7 carry the output cascode current $I_{cas} = I_{D1}$ and are in the signal path. Their intrinsic gain g_{m6}/g_{ds6} (field `gm_gds` in the LUT output) multiplies r_{o1} to form $R_{out,n}$.

- Use NMOS. Choose $L = 1.0\,\mu$m or longer if `gm_gds` is too low.
- Choose $g_m/I_D \approx 15\,\text{V}^{-1}$.
- For V_{DS6}: the cascode transistor is biased so that M1 is just at the edge of saturation, meaning $V_{DS6} \approx V_{dsat6}$. This maximizes output swing. Use $V_{DS6} = 0.2$ V as an initial approximation and refine after checking V_{dsat6} from the query output.

```
python query_lut.py \
  --l 1.0 --gmid 15 --vds 0.2 \
  --id <ID1>
```

Check: Inspect the gm_gds value. If $R_{out,n} = (\texttt{gm_gds})_6 \times r_{o1}$ combined with $R_{out,p}$ does not give enough gain (Step 8), increase L and re-query.

Record V_{GS6}, $W_{6,7}$, V_{dsat6}, and gm_gds.

6) Step 6: Size the PMOS Cascode Transistors (M8, M9): M8/M9 are the mirror image of M6/M7 on the PMOS side and carry the same current $I_{cas} = I_{D1}$.

- Use the PMOS LUT.
- Match the g_m/I_D and L choices made for M6/M7.
- Use $V_{SD8} = 0.2\,\mathrm{V}$ (same reasoning as Step 5).

```
python query_lut.py \
  --path char_data/pmos_lut.npz \
  --l 1.0 --gmid 15 --vds 0.2 \
  --id <ID1>
```

Record V_{SG8}, $W_{8,9}$, V_{dsat8}, and gm_gds.

7) Step 7: Estimate DC Gain: With all cascode transistors sized, estimate A_v to verify the 60 dB target before proceeding.

The output conductance of M1 is $g_{ds1} = g_{m1}/\texttt{gm_gds}_1$ (read gm_gds from the Step 2 query). Then:

$$R_{out,n} = (\texttt{gm_gds})_6 \times (1/g_{ds1}), \tag{16}$$

$$R_{out,p} = (\texttt{gm_gds})_8 \times (1/g_{ds4}), \tag{17}$$

$$R_{out} = R_{out,n} \| R_{out,p}, \tag{18}$$

$$A_v = g_{m1} \times R_{out}. \tag{19}$$

If $A_v < 1000\,\mathrm{V/V}$, increase the channel length of the cascode transistors (M6–M9) to raise gm_gds and re-query. Longer channels increase intrinsic gain but reduce f_T; the design space for L is shown in the LUT plots.

8) Step 8: Check Output Swing Headroom: Each cascode stack removes a minimum of two V_{dsat} values from the available output swing. The minimum output voltage is

$$V_{out,\min} = V_{dsat6} + V_{dsat1}, \tag{20}$$

and the maximum output voltage is

$$V_{out,\max} = V_{DD} - V_{dsat8} - V_{dsat4}. \tag{21}$$

Verify that $V_{out,\max} - V_{out,\min}$ meets the $\pm 1\,\mathrm{V}$ swing target around mid-rail ($1.65\,\mathrm{V}$ for a $3.3\,\mathrm{V}$ supply). If not, reduce the g_m/I_D of the cascode and folding devices toward strong inversion to shrink their V_{dsat} values and re-run Steps 4–6.

9) Step 9: Size the Class-AB Output Stage (M10, M11): The class-AB output stage is a push-pull pair (M10 NMOS, M11 PMOS) driven directly by the folded-cascode output. The class-AB bias circuit sets a quiescent current I_q through both devices while allowing them to source or sink much larger currents under a large signal.

a) Determine I_q:: The quiescent transconductance of the output stage determines the high-frequency output impedance seen by C_L. A common starting point is to set $g_{m,out} \approx 3$–$5 \times g_{m1}$ so that the output stage pole ($f \approx g_{m,out}/(2\pi C_L)$) is well above the unity-gain frequency. Choose I_q to achieve this.

b) Size M10 (NMOS)::

- Use NMOS LUT. Choose $g_m/I_D \approx 10\,\mathrm{V}^{-1}$ (strong inversion for low V_{dsat} to maximize output swing).
- Choose $L = 0.7\,\mu\mathrm{m}$ (shorter for lower V_{dsat}, but not minimum length to avoid poor r_o).
- Assume $V_{DS10} \approx V_{DD}/2$ at quiescent.

```
python query_lut.py \
  --l 0.7 --gmid 10 --vds 1.65 \
  --id <Iq>
```

c) Size M11 (PMOS):: Run the same query against the PMOS LUT with the same g_m/I_D target and I_q:

```
python query_lut.py \
  --path char_data/pmos_lut.npz \
  --l 0.7 --gmid 10 --vds 1.65 \
  --id <Iq>
```

Record V_{GS10}, V_{SG11}, W_{10}, W_{11}, and both V_{dsat} values.

10) Step 10: Determine Bias Voltages: Each transistor's V_{GS} (or V_{SG} for PMOS) returned by the LUT tells you the gate voltage required for that device to carry its target current. Use these to set the fixed bias voltages in the schematic:

- **M3 gate** (V_{b3}): set so $V_{b3} = V_{GS3}$.
- **M6/M7 NMOS cascode gate** (V_{b6}): the cascode gate must be high enough that M6/M7 are saturated while M1/M2 are also saturated. A starting value is $V_{b6} = V_{dsat1} + V_{GS6}$ (bias M6 so that when M1's drain sits at V_{dsat1}, M6's $V_{GS} = V_{GS6}$).
- **M8/M9 PMOS cascode gate** (V_{b8}): by symmetry, $V_{b8} = V_{DD} - V_{dsat4} - V_{SG8}$.
- **M4/M5 PMOS folder gate** (V_{b4}): set so $V_{DD} - V_{b4} = V_{SG4}$.

In practice these voltages are generated by a bias network (e.g., a diode- connected transistor or a simple resistor divider). At this stage, enter the calculated numerical values directly as voltage sources in LTspice to verify the operating point before adding a proper bias network.

11) Step 11: Verify in LTspice: Place all transistors in the schematic using the widths and lengths from Steps 2–9. Apply the bias voltages from Step 10. Run a .op simulation and check:

1) Each transistor is in saturation: $V_{DS} > V_{dsat}$ (NMOS) or $V_{SD} > V_{dsat}$ (PMOS).
2) The drain currents match the current budget from Section VIC.
3) The output node DC voltage sits near mid-supply when both inputs are at the same voltage (common mode).

If a transistor is in the linear region, adjust its bias voltage or widen the device to reduce V_{dsat} and re-run. Once the .op point looks correct, run an .ac analysis and verify GBW against (12).

G. Summary Sizing Table

Use Table III to record results as you work through the steps.

TABLE III
TRANSISTOR SIZING SUMMARY

Device	Type	g_m/I_D ($\mathbf{V}^{-1}$)	L (μm)	W (μm)	I_D (μA)	V_{dsat} (V)
M1, M2	NMOS					
M3	NMOS					
M4, M5	PMOS					
M6, M7	NMOS					
M8, M9	PMOS					
M10	NMOS					
M11	PMOS					

VII. REPRESENTATIVE LUT PLOTS

Four plots generated by `tiwari_plots.py` illustrate the NMOS design space. All curves are parametrized by channel length $L \in \{0.35, 0.70, 1.00, 2.00\}\,\mu$m.

Fig. 2 — I_D/W vs. gm/Id: This is the primary sizing chart used in (4). Given a g_m/I_D target, read the current density I_D/W from the vertical axis. Dividing the required drain current by this density gives the transistor width directly. Shorter channels carry more current per micron at the same g_m/I_D target, which means a narrower device but at the cost of lower intrinsic gain.

Fig. 3 — g_m/g_{ds} vs. gm/Id: This plot shows the intrinsic gain of a single transistor as a function of inversion level. Use it during Step 7 to estimate the cascode output resistance. Longer channels provide substantially higher intrinsic gain; the $L = 2.0\,\mu$m curve peaks near 650 V/V at moderate inversion.

Fig. 4 — $g_m r_o$ vs. V_{DS}: Intrinsic gain also depends on V_{DS}. This plot (at fixed $V_{ov} = 200\,$mV) shows that gain peaks around $V_{DS} \approx 2\,$V and falls at low V_{DS} (where the device is near the triode boundary) and at high V_{DS} (where channel-length modulation increases g_{ds}). Use it to verify that the chosen V_{DS} for each cascode transistor is in the high-gain region.

Fig. 5 — f_T vs. gm/Id: Use this to verify that the transistors are fast enough for the target GBW. As a rule of thumb, aim for $f_T > 10 \times$ GBW for each transistor in the signal path to keep parasitic poles well above unity gain. f_T peaks in moderate inversion and drops rapidly toward weak inversion.

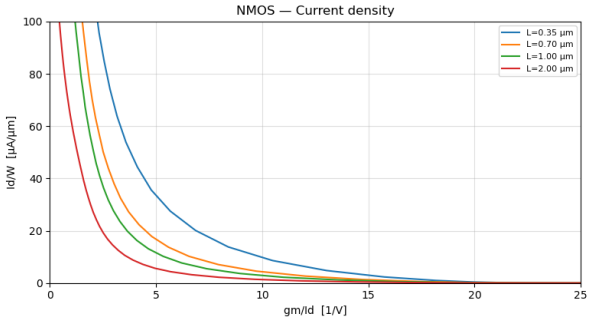

Fig. 2. Current density I_D/W vs. g_m/I_D for four NMOS channel lengths. Read the current density at the chosen g_m/I_D target, then apply (4) to find width.

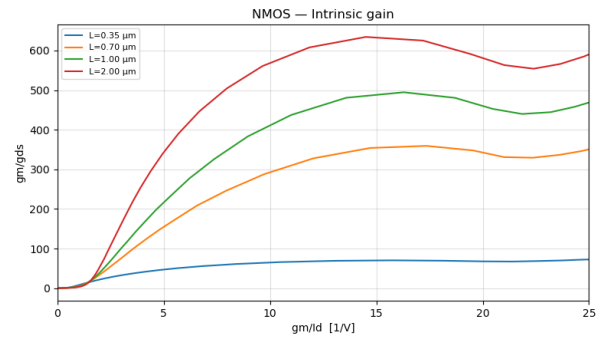

Fig. 3. Intrinsic gain g_m/g_{ds} vs. g_m/I_D for four NMOS channel lengths. Longer channels yield higher gain but reduce f_T.

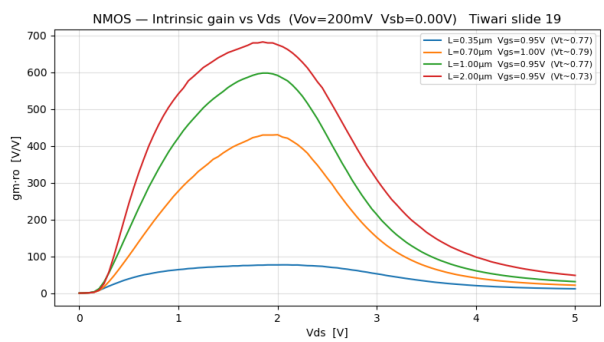

Fig. 4. Intrinsic gain $g_m r_o$ vs. V_{DS} at $V_{ov} = 200\,$mV, $V_{SB} = 0$. Gain peaks near $V_{DS} \approx 2\,$V and falls at both extremes.

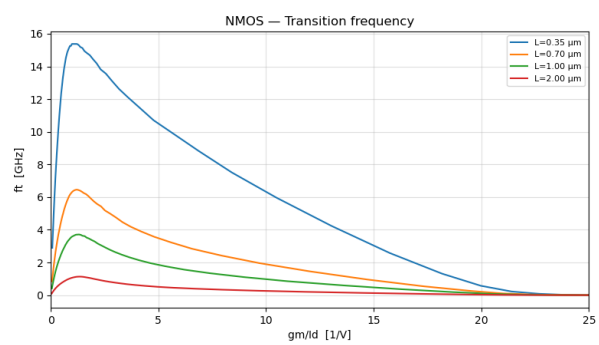

Fig. 5. Transition frequency f_T vs. g_m/I_D. Higher f_T requires stronger inversion (lower g_m/I_D) and draws more quiescent current. Minimum-length devices are fastest.

VIII. Conclusion

A Python-based gm/Id characterization and query tool has been presented. The tool automates the characterization sweep, stores results in a four-dimensional LUT, and provides a design-mode query that converts a g_m/I_D target and required current into a transistor width in a single step. The methodology was demonstrated through an eleven-step sizing procedure for a folded-cascode class-AB operational amplifier. Because all sizing data come directly from the SPICE model, the results are more accurate than square-law estimates, and no hand-calculation iteration is required before running the verification simulation.

Acknowledgment

The author thanks the Department of Electrical Engineering at Tarleton State University for providing access to simulation resources.

References

[1] F. Silveira, D. Flandre, and P. G. A. Jespers, "A gm/id based methodology for the design of CMOS analog circuits and its application to the synthesis of a silicon-on-insulator micropower OTA," *IEEE Journal of Solid-State Circuits*, vol. 31, no. 9, pp. 1314–1319, Sep. 1996.

[2] B. Murmann, "EE214B/EE315B lecture notes: gm/Id-based design methodology," Stanford University, Stanford, CA, 2020, [Online]. Available: https://web.stanford.edu/~murmann/.

[3] A. Tiwari, "Design of MOS amplifiers using gm/Id methodology," Slide deck, 2023, slides 14, 16, 17 referenced for design curves.

[4] P. E. Allen and D. R. Holberg, *CMOS Analog Circuit Design*, 3rd ed. New York, NY: Oxford University Press, 2011.

[5] N. Hoki, "PyLTSpice: A Python interface for LTspice simulation," GitHub, 2023, [Online]. Available: https://github.com/nunobrum/PyLTSpice.

[6] B. Razavi, *Design of Analog CMOS Integrated Circuits*. New York, NY: McGraw-Hill, 2001.